

STAT 5243 Project 2 — Interactive Data Workbench

1. Overview and Deployment

Our application is an interactive data workbench built with **Shiny for Python**. It lets users load, clean, transform, and explore tabular datasets entirely in the browser — no coding required. The interface is organized into five tabs (**Guide**, **Load**, **Cleaning**, **Feature Engineering**, **EDA**) accessible from a dark top navigation bar styled with the elegant Lux Bootstrap theme. All computation runs locally; there is no REST API or external backend, so user data never leaves the machine.

Deployed application: <https://019d23ea-1266-cada-1d21-45e5d97e6ea5.share.connect.posit.cloud/>

To run locally: `pip install -r requirements.txt` then `shiny run app.py` and open `http://127.0.0.1:8000`.

The app accepts **CSV**, **Excel (.xlsx/.xls)**, **JSON**, and **RDS** uploads (with robust error messages for corrupt or unsupported files) and includes three built-in datasets: **Sleep**, **Mobile and Stress** (15,000 rows of health survey data), **Iris** (150 rows of flower measurements), and **Tips** (244 rows of restaurant tipping data). These span a range of sizes and column types so every feature can be demonstrated out of the box.

2. Functionalities

Data Loading and Version History. Users load data in the **Load** tab by selecting a built-in dataset or uploading a file. The app immediately displays a summary card with row count, column count, missing-value count, and duplicate-row count. Every dataset loaded or derived from a cleaning/engineering step is saved in an **in-memory version history** (visible as a table below the summary card), so users can switch back to any previous version at any time using the dataset-picker dropdown.

Data Cleaning and Preprocessing. The **Cleaning** tab uses a sidebar layout: controls on the left, results on the right. Seven operations are available — handle missing values (six strategies: drop rows, drop columns, fill with mean, median, mode, or a constant), inspect and remove duplicates, scale numeric columns (standard, min-max, or robust scaling), encode categorical columns (label or one-hot), detect and handle outliers (statistical fence method with adjustable sensitivity — 1.5 for mild outliers, 3.0 for extreme), standardize text (trim whitespace and normalize letter case), and coerce column types. The column selector **automatically filters** to show only the relevant type for each operation (e.g., only numeric columns appear when scaling is selected). Users always click **Preview** first to see a before/after comparison chart and data table, then **Apply** to commit. Every control has a tooltip explaining what it does.

Feature Engineering. The **Feature Engineering** tab offers eleven transforms: log, square, cube, interaction (multiply two columns), ratio (divide with zero-denominator protection), binning, one-hot encoding, standardize (z-score), normalize (0-1 scale), fill missing values, and drop missing rows. When a method is selected, an **explanation panel** appears describing the transform in plain English (e.g., “Applies $\log(1+x)$. Reduces right-skew and compresses large values.”). After clicking **Preview**, the app shows the applied formula (e.g., `log_age = log1p(age)`), before/after summary statistics (mean and standard deviation), and a side-by-side distribution chart. A high-cardinality guard warns if one-hot encoding would create more than 50 columns.

Exploratory Data Analysis. The **EDA** tab provides summary tables (data preview with adjustable row count, descriptive statistics, column types), a free-text query filter (e.g., `age > 30 and gender == "Female"`), and five visualization panels — all rendered with Plotly for interactive zoom, pan, and hover:

- **1D plots** — histograms or bar charts with log-scale and normalization toggles, plus a persistent statistics panel showing mean, median, standard deviation, skewness, and kurtosis.
- **2D plots** — scatter, line, bar, box, heatmap, or 2D histogram. Column dropdowns show type labels like `age (num)` and `gender (cat)` to guide valid selections.
- **Regression** — polynomial fit (order 1 to 5), robust regression, or LOWESS (a smooth non-parametric curve). The plot displays **Pearson r**, **R-squared**, and **p-value** directly on the figure.
- **Multiline plots** — grouped distributions with shared bin edges for direct comparison.
- **Correlation matrix** — heatmap of pairwise correlations using Pearson, Spearman (rank-based), or Kendall (concordance-based) methods.

Export. CSV download buttons on the Load, Cleaning, and Feature Engineering tabs let users save the active dataset or any preview result at any stage.

3. How to Use the App

1. **Load a dataset.** Click the **Load** tab in the top navbar. Select a built-in dataset from the dropdown and click **Load Built-in Dataset**, or click the upload area, browse for your file, and click **Load Uploaded File**. The right-hand summary card will display row count, column count, missing values, and duplicates.
2. **Inspect your data.** Below the summary card, the **Dataset History** table shows every version you have created. Switch to the **EDA** tab to see the first rows (**Data Preview** card — adjust the row count with the numeric input), full descriptive statistics (**Describe** card), and a column-type breakdown (**Column Types** card).
3. **Clean and preprocess.** In the **Cleaning** tab sidebar, choose an operation from the **Action** dropdown (e.g., “Scale numeric columns”). The column list updates to show only relevant columns. Configure parameters (e.g., select “Robust” scaling), then click **Preview**. A before/after distribution chart appears on the right alongside a preview table. Click **Apply** to commit, or adjust and preview again. You can chain multiple operations — each creates a new version in the history.
4. **Engineer features.** In the **Feature Engineering** tab, select a method (e.g., “Log transform”), read the explanation that appears, and choose a primary column. Optionally set a custom output name (default: `log_age`). Click **Preview** to see the formula, before/after statistics, and comparison chart. Click **Apply** to add the column to your dataset.
5. **Explore with EDA.** In the **EDA** tab, type a filter expression in the query box (e.g., `stress_level > 7`) and click **Apply Filter**. Then render plots: pick a column and click **Render 1D Plot** to see its distribution with live statistics, or choose X and Y columns for a 2D plot. Use the **Regression** panel to fit a trend line and see R-squared. Generate a **Correlation Matrix** heatmap from the bottom panel to identify the strongest variable relationships.
6. **Download results.** Click any **Download CSV** button (on Load, Cleaning, or Feature Engineering tabs) to export your current dataset or preview to a local file.

4. Team Contributions

Team Member	Contribution
Cecilia Zang	Data cleaning and preprocessing backend (<code>data_cleaning.py</code>): 7 operations with validation, error handling, and pipeline support
Baixuan Chen	Feature engineering backend (<code>feature_engineering.py</code>): 11 transforms with formula tracking, metadata, and input validation
Yuhan Guo	EDA backend (<code>eda.py</code>): summary functions, filtering, 5 plot families, regression analysis, and correlation matrix
Zeming Liang	Shiny UI and integration (<code>app.py</code>): application assembly, reactive wiring, Lux theme, tooltips, deployment, testing, and report

5. Limitations and Future Work

- **Session-local state.** Version history is stored in server memory and lost when the app restarts; there is no database or file-based persistence.
- **Single-user design.** Concurrent users would share the same reactive state. A production deployment would need per-session isolation.
- **Future enhancements.** Deploy to a hosted platform (e.g., ShinyApps.io or Posit Connect Cloud), add violin and KDE density plots, and support the Parquet format for large-dataset workflows.